

Modélisation algorithmique complète

Optimisation des formations de coordonnateurs communaux - EAR 2027

Projet cc-formation-optimizer

30 juin 2026

Table des matières

1. Finalite du document	3
2. Vue d'ensemble du pipeline	3
3. Données d'entrée et préparation	4
4. Ensembles du modèle	4
5. Paramètres métier	5
6. Variables de décision	6
7. Fonction objectif	7
8. Contraintes	8
8.1 Affectation unique	8
8.2 Pas d'affectation sans ouverture	8
8.3 Capacité et remplissage minimal	8
8.4 Cohérence entre type et ouverture	9
8.5 Budget PC	9
8.6 Budget TPC	9
8.7 Ordre d'ouverture des slots	9
8.8 Interdiction des PC dans une formation TPC	9
8.9 Definition de la mixité résiduelle	10
9. Fonctionnement interne de CP-SAT et justification du paramétrage	10
9.1 Nature du solveur CP-SAT	10
9.2 Transformation interne du modèle	11
9.3 Propagation, conflits et Lazy Clause Generation	11

9.4 Recherche parallèle et amélioration des solutions	11
9.5 Paramètres solveur actuels	12
9.6 Paramètres métier actuels	12
9.7 Coûts d'éligibilité actuels	13
9.8 Justification des poids de l'objectif	14
9.9 Lecture opérationnelle des sorties CP-SAT	15
12. Assouplissement hiérarchique	15
13. Exports et visualisation	16
14. Performance et taille du modèle	16
15. Correspondance code / mathématiques	17
16. Limites et points de vigilance	17
17. Conclusion	18
Bibliographie	18

1. Finalite du document

Ce document décrit l'algorithme complet utilisé par l'outil d'optimisation des sessions de formation des coordonnateurs communaux PC/TPC pour l'EAR 2027. Il sert de référence pour comprendre le lien entre les données d'entrée, la construction du modèle CP-SAT, la résolution, la validation et les exports.

Le texte décrit l'état reel du code. Une règle, une contrainte ou un workflow n'est présente comme existant que s'il est effectivement implémenté. Les extensions non codees sont signalees comme limites ou perspectives.

Trois niveaux doivent être distingués pendant toute la lecture :

- les contraintes dures, qui definissent la faisabilité mathématique ;
- les pénalités de l'objectif, qui orientent le choix entre solutions faisables ;
- les contrôles de diagnostic, de validation, d'export et de carte, qui aident à qualifier une solution mais ne remplacent pas l'expertise métier.

2. Vue d'ensemble du pipeline

Le pipeline applique par le projet suit une chaîne volontairement séparée en étapes. Chaque étape produit des objets ou fichiers réutilisables par l'étape suivante, ce qui rend le calcul plus contrôlable et facilite l'audit.

```
Donnees brutes
-> donnees preparees
-> parametres derives
-> diagnostic pre-resolution
-> modele CP-SAT
-> resolution
-> extraction de solution
-> validation
-> exports
-> carte HTML
```

En pratique, la séquence opérationnelle est la suivante :

1. Charger la configuration YAML.
 2. Préparer les donnees brutes si necessaire.
 3. Charger les CSV propres.
 4. Construire les ensembles, couts et relations admissibles.
 5. Executer les diagnostics pre-resolution.
 6. Construire le modele CP-SAT.
 7. Resoudre le modele.
 8. Si le solveur trouve une solution :
 - extraire la solution metier ;
 - valider la solution extraite ;
 - produire les exports et, si demandé, la carte.
- Sinon :
- analyser le statut solveur ou utiliser le workflow d'assouplissement.

Cette séparation est importante : le solveur manipule des variables CP-SAT, tandis que les exports manipulent une solution métier reconstruite et validée. Les fichiers de restitution ne sont donc pas la source initiale de validation ; ils sont produits après la validation.

3. Données d'entrée et préparation

Le solveur ne lit pas directement les fichiers bruts. Les données réelles sont d'abord préparées par la commande `prepare-data`, qui transforme les fichiers source en tables propres exploitées par les modules de chargement.

Le fichier des communes fournit les identifiants INSEE, les noms, la population, la catégorie PC ou TPC, et des champs optionnels comme le territoire EAR, le nombre de logements, la latitude et la longitude. Ces données permettent de calculer le nombre de coordonnateurs communaux à former et de connaître la catégorie métier de chaque commune.

Le fichier des temps de trajet fournit des temps orientés entre une commune origine et une commune candidate pivot. Un trajet absent est traité comme interdit : il ne crée pas de variable d'affectation. La préparation ne complète pas automatiquement les trajets diagonaux $i \rightarrow i$; si un trajet diagonal doit être admissible avec un temps nul, il doit être présent dans les données propres.

Le fichier de coordonnées est optionnel pour l'optimisation. Lorsqu'il est présent, ses latitudes et longitudes sont jointes aux communes et servent à la carte HTML. Une commune sans coordonnées peut tout de même être affectée et apparaître dans les exports non cartographiques.

Le fichier de compatibilités est également optionnel. S'il est absent, le code considère que toutes les compatibilités métier valent 1 par défaut. S'il est présent, il permet d'interdire certains couples commune-pivot avant même la création des variables d'affectation.

Les principales sorties de préparation sont :

- `data/processed/communes_clean.csv` ;
- `data/processed/temps_trajet_clean.csv` ;
- `data/processed/compatibilites_clean.csv` si une source de compatibilités est disponible.

4. Ensembles du modèle

On note d'abord l'ensemble des communes à affecter :

$$C = \text{ensemble des communes à affecter}$$

Cet ensemble contient toutes les communes chargées depuis le fichier propre des communes.

Les communes PC et TPC forment deux sous-ensembles de C :

$$P \subset C$$

$$T \subset C$$

Le code suppose que chaque commune appartient à une seule catégorie :

$$C = P \cup T \quad P \cap T = \emptyset$$

Ici, T designe l'ensemble des communes TPC. Il ne faut pas le confondre avec le paramètre de temps maximal, également noté T dans la configuration. Le contexte indique toujours s'il s'agit d'un ensemble ou du seuil de trajet.

Toutes les communes sont candidates pivot :

$$F = C$$

Cela signifie que le modèle autorise toute commune chargée à héberger une session potentielle, sous réserve des autres contraintes et pénalités. Cette égalité ne signifie pas qu'une commune pivot ouverte est automatiquement membre de sa propre session : le modèle actuel ne force pas cette auto-affectation.

Les sessions candidates sont indexées par une commune pivot j et un rang de slot m :

$$S = \{(j, m) : j \in F, m \in \{1, \dots, M_j\}\}$$

Le nombre de slots dépend de la catégorie de la commune pivot :

$$M_j = \begin{cases} 3 & \text{si } j \in P, \\ 1 & \text{si } j \in T. \end{cases}$$

Une commune PC peut donc porter jusqu'à trois sessions candidates, tandis qu'une commune TPC ne peut porter qu'une session candidate. Ces slots sont seulement des possibilités : une session candidate devient réelle uniquement si sa variable d'ouverture vaut 1.

5. Paramètres métier

On note p_i la population de la commune i . Le nombre de coordonnateurs communaux à former dépend de cette population :

$$q_i = \begin{cases} 1 & \text{si } p_i \leq 5000, \\ 2 & \text{si } p_i > 5000. \end{cases}$$

Cette règle est codée dans `parameters.py` et validée par la configuration. Elle permet de raisonner en chargé de formation, pas seulement en nombre de communes.

Le temps de trajet de la commune i vers le pivot j est noté :

$$\tau_{ij} \geq 0$$

Les temps sont orientés : un trajet $i \rightarrow j$ et un trajet $j \rightarrow i$ sont deux entrées distinctes si les données les fournissent.

Le seuil maximal de trajet configuré est noté T . Un couple commune-pivot est admissible seulement si le trajet existe et respecte ce seuil :

$$a_{ij} = \begin{cases} 1 & \text{si le trajet } i \rightarrow j \text{ existe et } \tau_{ij} \leq T, \\ 0 & \text{sinon.} \end{cases}$$

Un trajet absent donne donc $a_{ij} = 0$. Il ne s'agit pas d'une forte pénalité : le couple est retiré du modèle d'affectation.

La compatibilité métier externe est notée :

$$b_{ij} \in \{0, 1\}$$

Si aucun fichier de compatibilité n'est fourni, l'implémentation pose :

$$b_{ij} = 1 \quad \forall i \in C, \forall j \in F$$

Toutes les compatibilités sont alors autorisées par défaut. Si un fichier est fourni, une valeur 0 interdit le couple correspondant.

Les autres paramètres métier structurants sont :

Paramètre	Signification
T	temps maximal admissible pour une affectation commune-pivot
Q	capacité maximale d'une session, en nombre de CC
L	remplissage minimal d'une session ouverte
f	budget maximal de sessions PC
k	budget maximal de sessions TPC
B	budget total déclaré, avec $B = f + k$
w_t	poids du temps de trajet dans l'objectif
w_e	poids des coûts d'éligibilité dans l'objectif
w_m	poids de la mixité résiduelle dans l'objectif
e_j^{PC}	coût d'éligibilité si le pivot j héberge une session PC
e_j^{TPC}	coût d'éligibilité si le pivot j héberge une session TPC

Les coûts d'éligibilité sont des pénalités de l'objectif. Même lorsqu'une valeur est très élevée, elle n'interdit pas à elle seule une session ; elle rend simplement cette option très défavorable, sauf si le couple est interdit par une contrainte ou par l'absence de variable.

6. Variables de décision

La variable d'affectation indique si une commune est rattachée à une session candidate :

$$x_{ijm} \in \{0, 1\}$$

$$x_{ijm} = 1 \iff \text{la commune } i \text{ est affectée à la session } (j, m).$$

Cette variable n'existe pas pour tous les triplets possibles. Elle est créée uniquement pour les couples commune-pivot admissibles et compatibles.

La variable d'ouverture indique si une session candidate est effectivement ouverte :

$$y_{jm} \in \{0, 1\}$$

$$y_{jm} = 1 \iff \text{la session } (j, m) \text{ est ouverte.}$$

La variable de type distingue les sessions PC et TPC :

$$z_{jm} \in \{0, 1\}$$

$$z_{jm} = 1 \iff \text{la session } (j, m) \text{ est de type TPC.}$$

Lorsque $z_{jm} = 0$ et que la session est ouverte, elle est interprétée comme une session PC.

La variable de mixité résiduelle est entière :

$$d_{jm} \in \mathbb{N}$$

Elle mesure le nombre de CC TPC affectés à une session PC. Pour une session TPC, elle peut rester nulle, car la présence de communes TPC y est naturelle.

La condition de creation des variables d'affectation est :

$$a_{ij} = 1 \quad \text{et} \quad b_{ij} = 1.$$

Cette restriction est decisive pour la taille du modèle : un couple interdit ne devient pas une variable pénalisée, il n'existe tout simplement pas dans le modèle CP-SAT.

7. Fonction objectif

Le modèle minimise une somme pondérée de trois composantes : le coût de trajet, le coût d'éligibilité des pivots et la mixité résiduelle.

La composante de trajet est :

$$O_{\text{trajet}} = \sum_{i \in C} \sum_{(j,m) \in S} q_i \tau_{ij} x_{ijm}$$

Chaque temps de trajet est multiplié par le nombre de CC de la commune. Une commune de plus de 5000 habitants pèse donc deux fois plus qu'une commune à un seul CC.

La composante d'éligibilité est :

$$O_{\text{éligibilité}} = \sum_{(j,m) \in S} \left[e_j^{PC} y_{jm} + (e_j^{TPC} - e_j^{PC}) z_{jm} \right]$$

Si la session est ouverte et PC, alors $y_{jm} = 1$ et $z_{jm} = 0$, donc le coût vaut e_j^{PC} . Si elle est TPC, alors $z_{jm} = 1$, et l'expression vaut e_j^{TPC} . Cette écriture évite tout produit entre variables.

La composante de mixité est :

$$O_{\text{mixité}} = \sum_{(j,m) \in S} d_{jm}$$

Elle penalise la présence de CC TPC dans des sessions PC, sans l'interdire.

L'objectif global est :

$$\min \quad w_t O_{\text{trajet}} + w_e O_{\text{éligibilité}} + w_m O_{\text{mixité}}$$

La formulation est linéaire : chaque terme est une constante multipliée par une variable booléenne ou entière, ou une somme de tels termes. Le modèle reste donc compatible avec CP-SAT et ne contient pas de produit entre variables.

8. Contraintes

Les contraintes ci-dessous sont les contraintes dures du modèle. Une solution qui en viole une n'est pas faisable, quel que soit son coût objectif.

8.1 Affectation unique

Pour chaque commune, exactement une variable d'affectation doit être active :

$$\sum_{(j,m) \in S_i} x_{ijm} = 1 \quad \forall i \in C$$

S_i désigne les sessions candidates pour lesquelles une variable x_{ijm} existe pour la commune i . Cette contrainte garantit qu'aucune commune n'est oubliée et qu'aucune commune n'est affectée deux fois.

8.2 Pas d'affectation sans ouverture

Une commune ne peut être affectée qu'à une session ouverte :

$$x_{ijm} \leq y_{jm} \quad \forall x_{ijm}$$

Si $y_{jm} = 0$, alors toutes les affectations vers cette session doivent valoir 0.

8.3 Capacité et remplissage minimal

La charge d'une session ouverte doit rester entre L et Q :

$$Ly_{jm} \leq \sum_{i \in C} q_i x_{ijm} \leq Qy_{jm} \quad \forall (j, m) \in S$$

Si la session est fermée, $y_{jm} = 0$ et la charge imposée est nulle. Si elle est ouverte, elle doit respecter le remplissage minimal et la capacité maximale.

8.4 Cohérence entre type et ouverture

Une session fermée ne peut pas être déclarée TPC :

$$z_{jm} \leq y_{jm} \quad \forall (j, m) \in S$$

Cette contrainte donne un sens à z_{jm} uniquement lorsque la session existe.

8.5 Budget PC

Le nombre de sessions PC ouvertes ne doit pas dépasser f :

$$\sum_{(j,m) \in S} (y_{jm} - z_{jm}) \leq f$$

Pour une session ouverte PC, $y_{jm} - z_{jm} = 1$. Pour une session TPC, cette quantité vaut 0. La contrainte compte donc les sessions PC.

8.6 Budget TPC

Le nombre de sessions TPC ouvertes ne doit pas dépasser k :

$$\sum_{(j,m) \in S} z_{jm} \leq k$$

Le budget total B est valide en configuration par l'invariant $B = f + k$. Dans le modèle, les deux contraintes précédentes pilotent directement les budgets par type.

8.7 Ordre d'ouverture des slots

Pour les pivots PC, les slots doivent être ouverts dans l'ordre :

$$y_{j,m+1} \leq y_{jm} \quad \forall j \in P, \forall m \in \{1, \dots, M_j - 1\}$$

Cette contrainte réduit les symétries. Elle évite par exemple d'ouvrir le slot 2 d'un pivot PC alors que le slot 1 du même pivot est fermé. Elle ne s'applique pas aux pivots TPC, qui n'ont qu'un seul slot.

8.8 Interdiction des PC dans une formation TPC

Une commune PC ne peut pas être affectée à une session de type TPC :

$$x_{ijm} \leq 1 - z_{jm} \quad \forall i \in P, \forall (j, m) \in S_i$$

Si $z_{jm} = 1$, la session est TPC et toutes les affectations de communes PC vers cette session sont forcées à 0. En revanche, une commune TPC peut être affectée à une session PC ; cette situation est autorisée mais pénalisée par la mixité.

Point de vigilance : cette contrainte porte sur la catégorie des communes affectées, pas sur la catégorie du pivot. Dans le modèle actuel, une session TPC peut donc avoir une commune pivot PC si aucune autre contrainte ne l'interdit. Comme le pivot n'est pas automatiquement forcé à appartenir à sa propre session, cette situation n'implique pas qu'une commune PC soit affectée à une session TPC.

8.9 Définition de la mixité résiduelle

On note le nombre de CC TPC affectés à une session :

$$n_{jm}^T = \sum_{i \in T} q_i x_{ijm}$$

La variable d_{jm} doit couvrir la part TPC présente dans une session PC :

$$d_{jm} \geq n_{jm}^T - Qz_{jm}$$

$$d_{jm} \geq 0$$

Si la session est TPC, $z_{jm} = 1$ et $n_{jm}^T - Qz_{jm}$ est inférieur ou égal à zéro grâce à la capacité maximale. d_{jm} peut donc rester nul. Si la session est PC, $z_{jm} = 0$ et d_{jm} mesure le nombre de CC TPC places dans une session PC.

9. Fonctionnement interne de CP-SAT et justification du paramétrage

Cette partie décrit ce qui se passe entre la formulation mathématique précédente et la solution extraite par le code. Elle remplace les anciennes parties diagnostic, résolution et extraction par une lecture plus centrée sur le solveur CP-SAT, les paramètres actuels de `config_ear2027.yaml` et le rôle des poids de l'objectif. Elle s'appuie sur la documentation OR-Tools et sur des références relatives à CP-SAT et à la Lazy Clause Generation [1, 2, 3, 4, 5].

9.1 Nature du solveur CP-SAT

CP-SAT est le solveur de programmation par contraintes d'OR-Tools pour des modèles entiers. La documentation OR-Tools insiste sur un point structurant : les variables et les contraintes doivent être exprimées avec des entiers. Le modèle de ce projet respecte cette contrainte : les variables x , y et z sont booléennes, la variable d est entière, les temps de trajet sont en minutes entières et les pénalités sont des constantes entières.

Le solveur n'exécute pas une recherche gloutonne du type "prendre le plus proche pivot". Il reçoit un modèle global : toutes les contraintes d'affectation, de capacité, de budget, de type de session et de mixité sont présentes en même temps. CP-SAT cherche alors une affectation complète qui respecte toutes les contraintes dures, puis minimise l'objectif pondéré.

Le caractère entier est important pour l'interprétation des poids. Une augmentation de 1000 dans l'objectif n'est pas une probabilité ou un score flou : c'est une unité objective entière que le solveur compare exactement aux autres composantes. Le paramétrage doit donc être lu comme une hiérarchie numérique entre arbitrages métier.

9.2 Transformation interne du modèle

Le code construit un CpModel avec des variables et des contraintes linéaires. Avant la recherche principale, CP-SAT valide le modèle et applique une phase de présolve. Cette phase cherche à simplifier le problème sans changer l'ensemble des solutions valides : suppression de variables inutiles, resserrement de domaines, détection de contraintes redondantes, substitutions et propagation des implications déjà évidentes.

Dans ce projet, plusieurs choix aident le présolve :

- les variables x_{ijm} ne sont créées que pour les trajets admissibles et les compatibilités autorisées ;
- les couples sans trajet ou au-delà de T sont absents du modèle, au lieu d'être présents avec une très forte pénalité ;
- la contrainte d'ordre $y_{\{j,m+1\}} \leq y_{\{jm\}}$ supprime une partie des symétries entre slots d'un même pivot PC ;
- les budgets séparés PC et TPC bornent directement le nombre de sessions que le solveur peut ouvrir.

Le présolve ne remplace pas les diagnostics métier. Il travaille sur le modèle formel transmis à CP-SAT. Les diagnostics du code restent utiles avant l'appel au solveur pour signaler des communes sans pivot admissible, une incohérence de budget ou un risque de capacité insuffisante. En revanche, la preuve finale de faisabilité ou d'infaisabilité appartient au solveur.

9.3 Propagation, conflits et Lazy Clause Generation

CP-SAT combine des techniques de programmation par contraintes, de satisfiabilité booléenne et d'optimisation entière. La logique centrale est proche de la Lazy Clause Generation décrite dans les articles cités : les propagateurs de contraintes réduisent les domaines possibles ; lorsqu'une contradiction est rencontrée, le solveur produit une clause expliquant le conflit ; cette clause est apprise pour éviter de refaire la même erreur plus tard dans la recherche.

Dans le modèle EAR 2027, une décision locale peut produire beaucoup d'implications. Par exemple, si une session z_{jm} est déclarée TPC, toutes les variables x_{ijm} des communes PC vers cette session doivent rester à 0. Si une session est fermée, toutes les affectations vers elle sont impossibles. Si une session est presque pleine, les affectations restantes qui dépasseraient Q deviennent impossibles.

La propagation permet donc de couper des branches avant de tester une solution complète. L'apprentissage de clauses ajoute une mémoire de recherche : après un conflit, CP-SAT conserve une explication logique qui empêche de revenir vers une combinaison équivalente. C'est une des raisons pour lesquelles la formulation linéaire, entière et sans produit entre variables est préférable ici : elle donne au solveur des contraintes exploitables directement.

9.4 Recherche parallèle et amélioration des solutions

CP-SAT peut utiliser plusieurs travailleurs de recherche. Avec `num_workers` : 8, la configuration autorise huit workers. Ils ne changent pas le modèle mathématique ; ils diversifient la recherche, les heuristiques et les preuves sur la même formulation. Ce choix est adapté à un problème d'affectation combinatoire où trouver une première solution et prouver sa qualité peuvent avoir des difficultés différentes.

Le solveur travaille avec deux objectifs complémentaires :

- trouver rapidement une solution faisable respectant toutes les contraintes ;
- améliorer cette solution ou prouver qu’aucune solution de meilleur coût n’existe.

C’est pourquoi une solution FEASIBLE est exploitable mais pas optimale au sens mathématique. Elle respecte les contraintes, mais le solveur n’a pas démontré qu’elle est la meilleure possible dans le temps alloué. Une solution OPTIMAL ajoute cette preuve. Un statut INFEASIBLE signifie que CP-SAT a prouvé l’absence de solution sous les contraintes actuelles. Un statut UNKNOWN signifie que les limites de recherche ont été atteintes ou qu’aucune conclusion exploitable n’a été produite.

9.5 Paramètres solveur actuels

Les paramètres solveur de config_ear2027.yaml sont :

Paramètre	Valeur actuelle	Justification
time_limit_seconds	2400	limite de 40 minutes ; elle évite une exécution non bornée tout en laissant au solveur le temps de trouver une solution et d’améliorer la preuve
num_workers	8	exploite le parallélisme disponible pour diversifier la recherche CP-SAT sans modifier le modèle
random_seed	1	stabilise les décisions pseudo-aléatoires pour rendre les comparaisons de runs plus reproductibles
log_search_progress	true	conserve une trace exploitable de la progression, utile pour distinguer absence de solution, manque de temps et lenteur de preuve

La limite de temps est cohérente avec la documentation OR-Tools, qui recommande de borner la recherche pour garantir que le programme termine dans un délai raisonnable. Ici, 2400 secondes est un compromis opérationnel : assez long pour un modèle départemental ou régional dense, mais suffisamment borné pour rester auditable.

La graine random_seed : 1 ne rend pas tous les comportements parallèles strictement bit à bit identiques sur toutes les machines. Elle fixe toutefois la part pseudo-aléatoire contrôlable et permet de comparer plus proprement deux configurations proches. Les logs activés sont justifiés car le statut final seul ne suffit pas toujours à diagnostiquer un modèle combinatoire : l’évolution des bornes, des conflits et des solutions intermédiaires donne une information utile.

9.6 Paramètres métier actuels

Les paramètres métier actuellement utilisés sont :

Paramètre	Valeur actuelle	Rôle dans le modèle
T	75	temps maximal admissible entre une commune et son pivot
Q	14	capacité maximale d’une session en nombre de CC

Paramètre	Valeur actuelle	Rôle dans le modèle
L	6	remplissage minimal d'une session ouverte
B	55	budget total déclaré de sessions
f	45	budget maximal de sessions PC
k	10	budget maximal de sessions TPC
threshold_population	5000	seuil au-dessus duquel une commune compte pour deux CC
below_or_equal	1	nombre de CC pour une commune de population inférieure ou égale au seuil
above	2	nombre de CC pour une commune au-dessus du seuil
M_PC	3	nombre maximal de slots candidats pour un pivot PC
M_TPC	1	nombre maximal de slots candidats pour un pivot TPC

Le seuil T : 75 est un choix structurant : il retire du modèle tous les couples commune-pivot au-delà de 75 minutes. Cette valeur contrôle à la fois la faisabilité et la taille du modèle. Un seuil plus bas réduirait le nombre de variables x, mais augmenterait le risque de communes sans affectation. Un seuil plus haut rendrait davantage d'affectations possibles, mais élargirait la recherche.

La capacité Q : 14 et le remplissage minimal L : 6 encadrent les sessions ouvertes. Le rapport entre les deux laisse une marge réelle de composition des groupes : une session n'est pas ouverte pour un effectif marginal, mais elle peut accueillir des combinaisons de communes de tailles différentes. Le solveur peut ainsi arbitrer entre proximité, type de session et remplissage.

Les budgets f : 45 et k : 10 sont les contraintes directement utilisées pour compter les sessions PC et TPC. Le budget total B : 55 est cohérent avec $B = f + k$; il sert de garde-fou de configuration et de lecture métier. Cette séparation est préférable à un seul budget global, car une solution avec trop de sessions TPC ou trop de sessions PC peut être indésirable même si le total reste correct.

Le seuil de population à 5000 transforme la demande en nombre de coordonnateurs communaux à former. Les communes au-dessus de ce seuil pèsent 2 dans les capacités et dans l'objectif de trajet, ce qui évite de traiter une commune très peuplée comme une commune de faible charge.

Les slots M_PC : 3 et M_TPC : 1 traduisent une asymétrie métier. Une commune PC peut porter jusqu'à trois sessions candidates, alors qu'une commune TPC n'en porte qu'une. Cette limite évite que le solveur concentre artificiellement trop de sessions sur un même pivot TPC et donne plus de flexibilité aux pivots PC.

9.7 Coûts d'éligibilité actuels

Les coûts d'éligibilité configurés par bande de population sont :

Population du pivot	e_PC	e_TPC	Interprétation
1501 et plus	0	0	pivot pleinement favorable
1000 à 1500	100	50	pivot possible, légèrement pénalisé
500 à 999	500	150	pivot possible mais nettement moins souhaitable

Population du pivot	e_PC	e_TPC	Interprétation
0 à 499	1000000000	500	pivot PC pratiquement exclu par pénalité ; pivot TPC encore possible mais coûteux

Ces coûts ne sont pas des contraintes dures. Ils entrent dans l'objectif via w_e , donc ils orientent le solveur parmi les solutions faisables. La valeur 1000000000 joue le rôle d'une pénalité quasi interdite : elle n'empêche pas formellement une solution, mais elle la rend dominée sauf absence d'alternative dans les contraintes. C'est cohérent avec le workflow d'assouplissement, qui prévoit explicitement de remplacer les très grands coûts par une pénalité finie si cette étape est autorisée.

Les coûts TPC sont plus faibles que les coûts PC dans les petites bandes. Cela traduit une tolérance plus grande pour des pivots TPC de taille modérée, alors que l'ouverture d'une session PC dans une très petite commune est très fortement découragée.

9.8 Justification des poids de l'objectif

Les poids actuels sont :

Poids	Valeur actuelle	Composante
w_t	1	temps de trajet pondéré par le nombre de CC
w_e	1000	coûts d'éligibilité des pivots
w_m	500	mixité résiduelle TPC dans les sessions PC

Le poids w_t : 1 conserve le trajet comme unité de base. Une minute supplémentaire pour une commune à un CC coûte 1 ; pour une commune à deux CC, elle coûte 2. Ce choix rend la composante trajet lisible : elle mesure des minutes-personnes de formation, avant arbitrage avec les autres dimensions.

Le poids w_e : 1000 donne une priorité forte à l'éligibilité des pivots. Par exemple, utiliser un pivot PC de bande 1000-1500 ajoute $100 * 1000 = 100000$ unités d'objectif ; utiliser un pivot PC de bande 500-999 ajoute $500 * 1000 =$

1. Ces ordres de grandeur dépassent largement quelques dizaines de minutes de trajet. Le modèle préfère donc normalement un pivot plus éligible, même si cela impose un trajet un peu plus long, tant que le seuil dur T reste respecté.

Ce choix est justifié si l'éligibilité du pivot représente une préférence métier plus importante que l'optimisation fine des minutes. Il évite qu'une petite amélioration de trajet conduise à ouvrir des sessions dans des pivots jugés peu adaptés. En revanche, il faut lire les résultats en gardant cette hiérarchie en tête : le solveur optimise d'abord fortement la qualité des pivots, puis affine les trajets à l'intérieur de cette structure.

Le poids w_m : 500 pénalise chaque CC TPC placé dans une session PC. Cette pénalité vaut l'équivalent objectif de 500 minutes-personnes de trajet. Elle est donc suffisamment élevée pour éviter la mixité résiduelle quand une solution TPC propre existe, mais elle reste inférieure à une pénalité d'éligibilité moyenne pondérée par w_e . Ainsi, le modèle préfère généralement ouvrir et remplir des sessions TPC, sans rendre impossible l'affectation de TPC en session PC lorsque les contraintes de trajet, de capacité ou de budget ne permettent pas mieux.

L'ordre de priorité induit par les poids actuels est donc :

1. respecter toutes les contraintes dures, sans exception ;
2. éviter les pivots peu éligibles, surtout pour les sessions PC ;
3. limiter la mixité TPC résiduelle dans les sessions PC ;
4. minimiser les temps de trajet parmi les solutions qui satisfont ces arbitrages.

Cet ordre est cohérent avec une lecture métier où la solution doit d'abord être opérationnellement acceptable, puis géographiquement efficace. Les temps de trajet ne sont pas ignorés : ils départagent toutes les solutions comparables en éligibilité et en mixité. Ils sont simplement placés derrière les pénalités qui expriment les préférences structurelles du modèle.

9.9 Lecture opérationnelle des sorties CP-SAT

À la fin de la recherche, CP-SAT retourne un statut et des valeurs de variables. Les statuts utiles sont :

Statut	Interprétation opérationnelle
OPTIMAL	une solution faisable est trouvée et sa qualité optimale est prouvée
FEASIBLE	une solution faisable est trouvée, sans preuve d'optimalité
INFEASIBLE	aucune solution ne respecte les contraintes actuelles
MODEL_INVALID	le modèle transmis au solveur est invalide
UNKNOWN	le solveur s'arrête sans solution ni preuve suffisante, souvent à cause d'une limite

Le code ne doit exploiter une solution que lorsque le statut est OPTIMAL ou FEASIBLE. Les valeurs CP-SAT sont ensuite reconstruites en objets métier : sessions ouvertes, communes affectées, charges, types de session, temps de trajet et décomposition de l'objectif. Cette extraction reste nécessaire, car le solveur ne retourne que des valeurs de variables.

La validation post-solution reste une étape distincte. Elle recalcule les règles importantes à partir de la solution extraite : affectation unique, existence des sessions référencées, capacités, budgets, interdiction des PC en session TPC, temps de trajet, compatibilités, cohérence de la mixité et objectif recalculé. La condition opérationnelle reste donc :

$$\text{solution exploitable} \iff \text{statut CP-SAT exploitable} + \text{solution extraite} + \text{validation OK}$$

Les exports ne sont produits qu'après cette validation. Ils servent à l'analyse et à la restitution, mais ils ne remplacent ni la preuve du solveur ni le contrôle post-solution du code.

12. Assouplissement hiérarchique

Le workflow **solve-relaxed** applique une hiérarchie de tentatives. Il commence toujours par la configuration initiale. Si aucune solution validée n'est obtenue, il crée des copies indépendantes de la configuration et modifie uniquement des paramètres explicitement autorisés par le protocole.

Les niveaux implémentés sont :

1. configuration initiale ;
2. ajustement du poids w_m ;

3. reduction des coûts d'éligibilité TPC ;
4. augmentation du seuil de trajet T ;
5. reduction du remplissage minimal L ;
6. augmentation de la capacité Q ;
7. augmentation cohérente de f , k et B ;
8. remplacement final des coûts très élevés par une pénalité finie, si cette option est autorisée.

À chaque tentative, toute la chaîne est relancée : paramètres dérivés, modèle CP-SAT, solveur, extraction et validation. Le workflow s'arrête à la première solution validée et journalise les tentatives exécutées.

La contrainte interdisant les communes PC dans une session TPC n'est jamais relâchée automatiquement. Si cette contrainte bloque une zone, il s'agit d'une décision métier à examiner explicitement, pas d'un assouplissement standard du code.

13. Exports et visualisation

Une solution validée peut être restituée sous plusieurs formes :

Fichier	Rôle
<code>sessions.csv</code>	détail des sessions ouvertes, chargés, types, alertes et indicateurs
<code>communes_affectees.csv</code>	affectation de chaque commune à une session et à un pivot
<code>rapport_solution.md</code>	synthèse lisible de la solution et des contrôles
<code>statistiques_solution.json</code>	statistiques structurées pour analyse ou réutilisation
<code>config_utilisee.yaml</code>	copie de la configuration ayant produit la solution
<code>solution_map.html</code>	carte HTML autonome de contrôle visuel

La source de vérité opérationnelle est :

source de vérité = exports valides + configuration utilisée

La carte est une visualisation de contrôle. Elle aide à inspecter les pivots, les affectations, les temps de trajet proches du seuil et les alertes, mais elle ne remplace pas les CSV, le rapport, le JSON de statistiques et la configuration utilisée.

Les coordonnées n'interviennent pas dans le modèle CP-SAT. Elles servent uniquement à positionner les points sur la carte. Les communes sans coordonnées restent dans la solution et dans les exports.

14. Performance et taille du modèle

La taille du modèle dépend surtout du nombre de variables d'affectation x . Ces variables sont créées pour les couples commune-pivot admissibles, compatibles, puis dupliquées sur les slots du pivot.

$$|X| = \sum_{i \in C} \sum_{j \in F} M_j I_{a_{ij}=1} I_{b_{ij}=1}$$

Cette formule explique pourquoi le seuil de trajet T à un effet important. Dans cette notation, $I_condition$ vaut 1 si la condition est vraie et 0 sinon. Augmenter T augmente souvent le nombre de couples admissibles et donc le nombre de variables. Le problème peut devenir plus facile à rendre faisable, mais plus difficile à optimiser. Réduire T à l'effet inverse : le modèle est plus petit, mais le risque d'infaisabilité augmente.

Le nombre de pivots candidats et les valeurs M_j jouent aussi un rôle direct. Comme $F = C$, chaque commune est candidate pivot. Les communes PC génèrent jusqu'à trois slots, alors que les communes TPC n'en génèrent qu'un.

Il faut enfin distinguer trouver une solution faisable et prouver son optimalité. CP-SAT peut trouver rapidement une solution respectant les contraintes, puis avoir besoin de beaucoup plus de temps pour prouver qu'elle est optimale. Le parallélisme peut améliorer la recherche, mais il ne change ni les contraintes ni la fonction objectif.

15. Correspondance code / mathématiques

Element	Signification	Fichier
C, P, T, F, S	ensembles	<code>parameters.py</code>
q_i	nombre de CC	<code>parameters.py</code>
τ_{ij}	temps de trajet	<code>parameters.py</code>
a_{ij}	admissibilité trajet	<code>parameters.py</code>
b_{ij}	compatibilité métier	<code>parameters.py</code>
e_j_PC, e_j_TPC	coûts d'éligibilité	<code>parameters.py</code>
x, y, z, d	variables CP-SAT	<code>model_builder.py</code>
objectif	fonction à minimiser	<code>model_builder.py</code>
contraintes	modèle CP-SAT	<code>model_builder.py</code>
extraction	solution métier	<code>solution_extractor.py</code>
validation	contrôle post-solution	<code>validation.py</code>
assouplissement	scenarios hiérarchiques	<code>relaxation.py</code>
exports	restitution	<code>export.py</code>
carte	visualisation	<code>map_export.py</code>

16. Limites et points de vigilance

Le modèle actuel est volontairement centré sur l'affectation des communes aux sessions. Les limites suivantes doivent être connues avant toute utilisation métier :

- un statut FEASIBLE ne garantit pas l'optimalité ;
- les poids w_t , w_e et w_m doivent être calibrés avec prudence ;
- les coûts très élevés sont des pénalités, pas des interdictions ;
- les trajets absents sont interdits car aucune variable n'est créée ;

- les trajets diagonaux $i \rightarrow i$ ne sont pas générés automatiquement ;
- les coordonnées servent uniquement à la carte ;
- une commune pivot ouverte n'est pas forcément affectée à sa propre session ;
- une session TPC peut avoir un pivot PC si le pivot n'est pas contraint à être membre de sa session ;
- les superviseurs, disponibilités et contraintes calendaires ne sont pas optimisés si aucune variable ou contrainte dédiée ne les encode ;
- une validation métier finale reste nécessaire après la validation algorithmique.

Ces limites ne sont pas des erreurs d'exécution. Elles delimitent le périmètre exact du modèle implémenté et les points à discuter avant toute extension.

17. Conclusion

L'outil produit une solution d'affectation en combinant une préparation de données, une formulation CP-SAT, une résolution paramétrée, une extraction métier et une validation indépendante. Cette chaîne permet d'obtenir des exports contrôlés et une carte de visualisation sans confondre optimisation, validation et expertise métier.

La validation algorithmique garantit que la solution extraite respecte les contraintes codées. Elle ne remplace pas l'analyse humaine des alertes, des choix de pénalités, des pivots retenus et des conditions opérationnelles non modélisées. Les exports et la carte servent donc à qualifier la solution et à faciliter la revue métier.

Ce document constitue la référence pour comprendre comment les objets du code, les notations mathématiques et les fichiers produits s'articulent dans l'implémentation actuelle.

Bibliographie

Références

- [1] Google OR-Tools, *CP-SAT Solver*. https://developers.google.com/optimization/cp/cp_solver
- [2] Google OR-Tools, *Setting solver limits*. https://developers.google.com/optimization/cp/cp_tasks
- [3] Daniel Krupke, *The CP-SAT Primer*, chapitre *Under the hood*. https://d-krupke.github.io/cpsat-primer/under_the_hood.html
- [4] Ohrimenko, Stuckey et Codish, *Propagation via Lazy Clause Generation*. <https://doi.org/10.1007/s10601-009-9088-0>
- [5] Schutt, Feydy, Stuckey et Wallace, *Solving the Resource Constrained Project Scheduling Problem with Generalized Precedences by Lazy Clause Generation*. <https://arxiv.org/abs/1009.0347>